

# Text2CAD-Bench: A Benchmark for LLM-based Text-to-Parametric CAD Generation

Liang Wang<sup>\*12</sup> Heng Meng<sup>\*12</sup> Zekai Xiang<sup>\*23</sup> Jin Liu<sup>†1</sup> Pingyi Zhou<sup>†2</sup> Litao Chen<sup>2</sup> Yongqiang Tang<sup>4</sup>

## Abstract

Text-to-CAD generation aims to create parametric CAD models from natural language, enabling rapid prototyping and intuitive design workflows. However, existing benchmarks focus on basic primitives and simple sketch-extrude sequences, lacking advanced features essential for real-world applications and covering only traditional mechanical parts. We introduce Text2CAD-Bench, the first benchmark systematically evaluating text-to-CAD across geometric complexity and application diversity. Our benchmark comprises 600 human-curated examples spanning four levels: L1-L2 cover fundamental geometry with standard features, L3 introduces complex topology and freeform surfaces, and L4 extends to real-world domains beyond mechanical parts. Each example pairs dual-style prompts—geometric descriptions mimicking non-expert users, and procedural sequences aligned with expert-level conventions. Evaluating mainstream general LLMs and domain-specific models, we find that current models perform reasonably on basic geometry but degrade substantially on complex topology and advanced features. We release our benchmark to drive progress in text-to-CAD research.

## 1. Introduction

The emergence of Large Language Models (LLMs) has catalyzed significant progress in Computer-Aided Design (CAD), particularly in the automatic generation of parametric models from natural language. Unlike mesh or point

cloud representations, parametric CAD maintains editable construction histories and precise dimensional constraints, making it essential for engineering design and manufacturing. Early approaches, such as DeepCAD(Wu et al.), pioneered the field by employing sequence-based representations of sketch-and-extrude operations(Koch et al.). Recently, however, research has shifted toward code-based generation—using frameworks such as CadQuery or Python scripts (Xie & Ju). By capitalizing on the code synthesis capabilities of pre-trained LLMs(Li et al., 2023), code-based methods offer a higher level of abstraction, enabling more natural descriptions of complex geometries and parametric relationships compared to low-level command sequences.

Despite these advances, the systematic evaluation of text-to-CAD systems remains limited. Existing benchmarks exhibit three key shortcomings that lead to a misalignment between human design intent and machine-executable representation.

First, geometric complexity is heavily restricted: current datasets predominantly consist of simple sketch-extrude primitives and lack support for advanced modeling features such as chamfers, fillets, sweeps, lofts, and freeform surfaces. As a result, generated models tend to be geometrically simplistic and often inadequate for real-world manufacturing.

Second, domain coverage is narrow, with prior benchmarks focusing almost exclusively on traditional mechanical parts and basic primitives, thereby overlooking diverse applications in areas such as medical devices, consumer products, and architectural fabrication.

Third, linguistic diversity is insufficient and structurally biased. Descriptions in existing benchmarks frequently mirror low-level command sequences rather than convey high-level design intent. This encourages models to learn a superficial “command-translation” mapping instead of achieving genuine “shape-understanding,” ultimately failing to capture how humans naturally describe shapes—whether through visual-geometric attributes or procedural construction logic.

To bridge these gaps, we introduce Text2CAD-Bench, the first comprehensive benchmark for evaluating text-to-CAD generation across a rigorously defined complexity hierarchy. Our benchmark consists of 600 human-curated exam-

<sup>\*</sup>Equal contribution. <sup>†</sup>Corresponding authors.

Liang Wang, Heng Meng, and Zekai Xiang completed this work during their internship at Spatial Design Intelligence Lab, BitInf Ltd.

<sup>1</sup>School of Computer Science, Wuhan University, Wuhan 430000, Hubei, China

<sup>2</sup>Spatial Design Intelligence Lab, BitInf Ltd., Shanghai 200003, China

<sup>3</sup>College of Computer and Information Engineering, Nanjing Tech University, Nanjing 211800, Jiangsu, China

<sup>4</sup>State Key Laboratory of Multimodal Artificial Intelligence Systems, Institute of Automation, Chinese Academy of Sciences, Beijing 100190, China

Correspondence to: Jin Liu <jin.liu@whu.edu.cn>, Pingyi Zhou <pingyi.zhou@bitinf.com>.

Preprint. May 19, 2026.

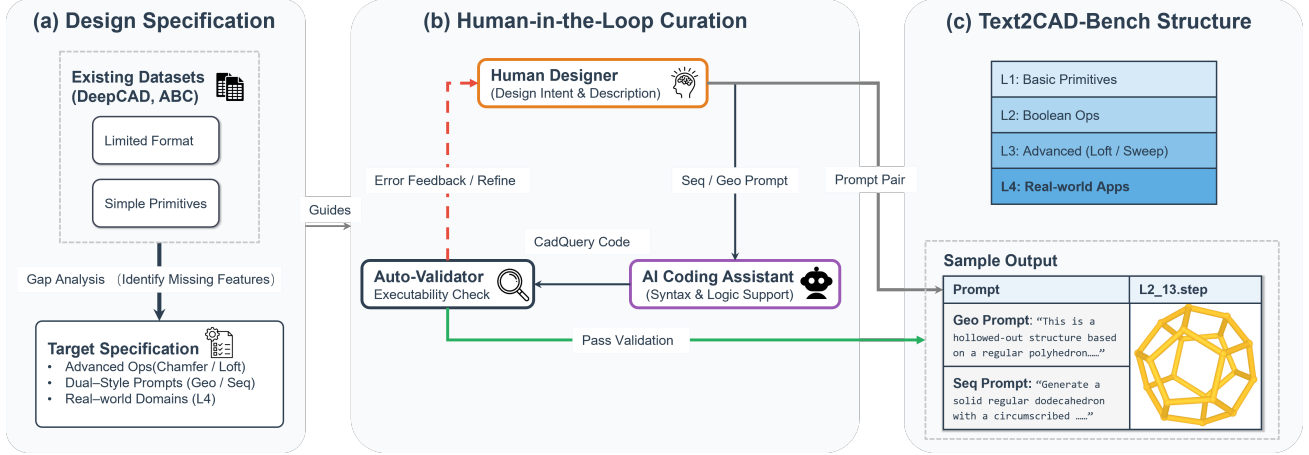


Figure 1. Overview of the Text2CAD-Bench construction pipeline. Our human-in-the-loop process comprises four stages: (1) Design Specification defines target geometry and complexity levels; (2) Collaborative Authoring combines human expertise with AI assistance to develop validated CadQuery code; (3) Description Generation creates dual-style prompts (geometric and sequence) for each model; (4) Quality Assurance ensures executability, description-geometry correspondence, and cross-style consistency.

ples, organized into four progressive tiers from L1 to L4. L1–L3 systematically evaluate geometric reasoning from basic primitives to advanced topological operations, such as sweeping a profile along a curved path. L4 extends the evaluation to diverse real-world domains, requiring models to interpret geometric specifications within concrete application contexts—for instance, designing a medical brace or a consumer electronics accessory. This structure not only tests a model’s ability to handle increasingly complex geometry, but also examines its generalization beyond conventional mechanical parts.

Our contributions are summarized as follows:

- We construct **Text2CAD-Bench**, a dataset of 600 high-quality parametric CAD models organized into four difficulty levels, specifically addressing the lack of advanced geometric operations (chamfers, fillets, sweeps, lofts, complex surfaces) and application diversity in prior benchmarks.
- We design dual-style prompts—geometric descriptions mimicking non-expert users and command sequences aligned with expert-level CAD conventions—and empirically demonstrate that geometric-style prompts yield superior generation performance across evaluated models.
- Through comprehensive evaluation of mainstream LLMs and domain-specific models, we reveal that current state-of-the-art systems struggle significantly on advanced features (L3) and real-world scenarios (L4), establishing Text2CAD-Bench as a challenging target for future research.

## 2. Related Work

### 2.1. CAD Generation Methods

Deep learning approaches to CAD generation have evolved significantly in recent years. DeepCAD (Wu et al.) pioneered the representation of CAD models as sequences of sketch-and-extrude operations, training a Transformer-based autoencoder on 178K models. This sequence-based paradigm was subsequently extended by SkexGen (Xu et al., b), which disentangled topology and geometry into separate codebooks to address posterior collapse, HNC (Xu et al., a) introduced hierarchical code trees enabling controllable generation and ExtrudeNet (Ren et al., 2022) focuses on the research of extrude operation.

An alternative line of work operates directly on boundary representations (B-rep) (Xu et al., 2024). SolidGen (Jayaraman et al.) proposed an autoregressive model that synthesizes B-rep geometry without requiring construction sequence supervision. Despite their progress (Seff et al., 2022), these methods share a common limitation: they require training specialized networks from scratch on task-specific representations, unable to leverage the vast knowledge encoded in pretrained language models.

### 2.2. Text-to-CAD Generation

Text-to-3D generation has witnessed rapid progress in recent years. Early work such as Text2Shape (Jun & Nichol, 2023) pioneered the mapping from natural language to 3D shapes using joint embeddings. More recently, DreamFusion (Poole et al., 2022) leveraged score distillation sampling to optimize Neural Radiance Fields (NeRF) from text prompts, enabling high-fidelity 3D content creation. Point-E (Nichol et al.) and Shap-E (Jun & Nichol, 2023) from Ope-

nAI further accelerated generation by directly producing point clouds and implicit representations, respectively.

However, these methods generate mesh or implicit representations that lack the parametric structure and dimensional constraints required for engineering applications. Text-driven CAD generation has emerged as a promising direction, with approaches categorized by their output representation into sequence-based methods and code-based methods.

Sequence-based methods generate task-specific command sequences. Text2CAD (Khan et al.) introduced the first large-scale text-CAD dataset by augmenting DeepCAD models with template-generated textual descriptions across four prompt levels. A Transformer decoder maps text embeddings to CAD command sequences, which are then converted to 3D models via a pretrained decoder.

Code-based methods leverage pretrained LLMs to generate executable CAD scripts. LLM4CAD (Li et al.) explored multimodal inputs for CadQuery code generation, finding that text-only input outperforms image-based or multimodal combinations. Text-to-CadQuery (Xie & Ju) demonstrated that fine-tuning LLMs on 170K text-CadQuery pairs yields consistent improvements with model scale. CAD-Coder (Hui et al.) further incorporated chain-of-thought reasoning and geometric reward-based reinforcement learning to enhance code validity. In the reverse engineering domain, CAD-Recode (Rukhovich et al.) showed that even small LLMs, when combined with a lightweight point cloud projector, can generate high-quality CadQuery code for CAD reconstruction.

We adopt CadQuery as our target representation for several reasons: (1) it directly leverages LLMs’ code generation capabilities; (2) its method-chaining API naturally aligns with natural language descriptions; (3) it supports advanced features (chamfers, fillets, sweeps, lofts) beyond sketch-extrude; and (4) generated code is immediately executable for validation.

### 2.3. CAD Datasets and Benchmarks

Several datasets have been proposed for CAD-related research. ABC (Koch et al.) provides one million CAD models with parametric surface representations but lacks construction sequences. DeepCAD (Wu et al.) and Fusion 360 Gallery (Willis et al.) include construction sequence information but are limited to sketch-extrude operations. Text2CAD (Khan et al.) is the only existing dataset with text annotations, but its descriptions are template-generated from underlying command sequences, lacking linguistic diversity and genuine alignment with human design intent.

Inspired by recent benchmarks that prioritize quality over quantity—such as HumanEval (Chen et al.), which uses

164 hand-crafted programming problems to evaluate code generation, MATH-500 (Hendrycks et al.), which employs expert-written problems to probe mathematical reasoning boundaries, and GAIA (Mialon et al.). Rather than scaling to hundreds of thousands of template-generated examples, we invest in 600 human-verified instances that systematically cover the geometric complexity spectrum (L1-L3) and diverse application domains (L4). Each example undergoes rigorous quality assurance to ensure unambiguous ground truth, correct classification, and faithful correspondence between text descriptions and CAD geometry. This design philosophy enables fine-grained diagnosis of model capabilities and limitations, which large-scale but homogeneous benchmarks often obscure.

## 3. Text2CAD-Bench

We present Text2CAD-Bench, a comprehensive benchmark for evaluating text-to-CAD generation capabilities. The benchmark comprises 600 human-curated examples organized into four benchmark levels, with each example paired with dual-style natural language descriptions.

### 3.1. Design Principles

Our benchmark design is guided by three core principles that address limitations identified in existing datasets.

**Unambiguous Ground Truth.** Each text description in our benchmark corresponds to a unique, fully-specified CAD model with all geometric parameters explicitly defined. Both the geometric description and sequence description for the same example describe identical geometry, ensuring that given the same input, only one correct output exists.

**Systematic Complexity Gradation.** Unlike Text2CAD (Khan et al.), whose benchmark levels primarily reflect textual detail (abstract to expert), we stratify examples by geometric complexity.

**Dual Description Styles.** Real users describe designs differently depending on their expertise and intent. We provide two complementary description styles: geometric descriptions that characterize shapes by their appearance and spatial relationships, and sequence descriptions that specify step-by-step construction sequences. This duality enables evaluation of model robustness to varying input formats and provides insights into which description style better facilitates CAD generation.

### 3.2. Benchmark Construction

Figure 1 illustrates our human-in-the-loop construction pipeline, comprising four stages designed to ensure quality, consistency, and coverage.

**Design Specification.** We analyzed the Text2CAD

dataset (Khan et al.) to identify core geometric patterns and coverage gaps. This analysis revealed that existing data predominantly contains sketch-extrude sequences with limited feature diversity. We extended coverage to include advanced CAD operations—chamfer, fillet, sweep, loft, shell, and complex patterns—essential in professional workflows but absent from current benchmarks. Additionally, we surveyed emerging application domains including medical devices, consumer products, and architectural fabrication to ensure real-world relevance.

**Collaborative Model Authoring.** Human designers specify the target geometry and benchmark level, then iteratively develop CadQuery code with AI assistance (Claude, GPT-4) until successful execution. This collaborative process leverages AI efficiency while maintaining human oversight for quality control. We enforce three validity constraints: (1) all geometric parameters must be fully specified; (2) the generated mesh must be valid and watertight; (3) geometric complexity must match the target benchmark level.

**Description Generation.** Each validated CAD model receives two natural language descriptions following carefully designed templates.

- **Geometric Descriptions.** We adopt a structured description pattern following the sequence: global shape → local features → fine details → global summary. Through preliminary experiments comparing multiple description strategies, we found this pattern yields superior generation results compared to alternatives such as purely sequential descriptions or detail-first approaches. A representative example:

*“A rectangular enclosure with rounded vertical edges. The top face features a circular opening positioned near one end. All vertical edges are finished with 2mm fillets. The circular opening has a diameter of 15mm, centered 20mm from the shorter edge. The overall dimensions are 80mm in length, 50mm in width, and 30mm in height.”*

- **Sequence Descriptions.** We extend the expert-level instruction sequence format introduced in Text2CAD (Khan et al.). Since the original format covers only sketch-extrude operations, we design analogous command-style descriptions for advanced features by referencing standard CAD command conventions, preserving the sequential nature while expanding feature coverage:

*“Create a workplane on the XY plane. Sketch a rectangle with dimensions 80mm × 50mm and corner radius 5mm. Extrude the sketch 30mm in the positive Z direction. Select the top face and create a new sketch. Draw a circle with diameter 15mm centered at offset (20mm, 25mm) from the face origin. Perform a cut-through*

*operation. Select all vertical edges and apply a fillet with radius 2mm.”*

### 3.3. Benchmark Levels

We define four benchmark levels to enable systematic evaluation across the spectrum of text-to-CAD challenges. Levels are determined by geometric complexity and feature distribution, with manual verification as a secondary check. Detailed classification criteria are provided in A. L1–L3 form a hierarchy based on geometric complexity, while L4 targets real-world application diversity.

#### 3.3.1. GEOMETRIC COMPLEXITY HIERARCHY

The Benchmark levels from L1 to L3 are stratified by the complexity of required CAD operations and geometric features, as summarized in Appendix A. The progression from L1 to L3 reflects increasing demands on the model’s geometric reasoning capabilities:

L1 tests fundamental understanding of primitive shapes and basic spatial relationships. Success at this level indicates the model can parse geometric descriptions and generate valid, simple CadQuery code.

L2 introduces compositional complexity through boolean operations and standard CAD features. Models must correctly sequence multiple operations and handle feature interactions such as applying fillets to edges created by boolean cuts.

L3 requires mastery of sophisticated CAD operations that many current models struggle with. Sweep and loft operations demand understanding of path-based geometry.

Basic finishing features such as chamfer and fillet appear across all levels, applied to geometries of corresponding complexity. Complex patterns and freeform surfaces are concentrated in L3 due to their inherent difficulty.

#### 3.3.2. REAL-WORLD APPLICATION SCENARIOS

L4 comprises 100 examples representing real-world design scenarios across diverse application domains. Unlike L1–L3, which focus on isolated geometric capabilities, L4 evaluates whether models can handle the contextual variety of practical text-to-CAD applications. We emphasize that L4 examples still provide precise geometric descriptions with fully specified parameters—they are not underspecified functional requirements. The distinction from L1–L3 lies in:

**Domain diversity:** Examples span industrial/mechanical parts (~40%), consumer products (~25%), medical devices (~15%), architectural elements (~10%), and educational models (~10%).



**Contextual richness:** Descriptions may reference application context or domain-specific conventions (e.g., “suitable for M4 screw”) that models must interpret correctly.

**Practical relevance:** Examples reflect actual design tasks that may be encountered in emerging text-to-CAD applications rather than abstract geometric exercises.

The geometric complexity of L4 examples varies—some may be simpler than typical L3 examples, while others are comparably complex. L4’s primary purpose is assessing generalization across application domains rather than pushing geometric difficulty limits.

**Scope restriction.** All L4 examples consist of a single solid body without assemblies, mating constraints, or hidden internal mechanisms. This restriction is deliberate: the VLM-based scoring protocol (Section 3.3.3) operates on 8 multi-view renderings of the exterior surface, and is therefore inherently limited in judging internal structures, precise dimensional constraints, or inter-part relationships. By confining L4 to single-body designs whose correctness is largely visible from external views, we keep the evaluation tractable and the VLM judgments interpretable.

### 3.3.3. EVALUATION METRICS

We adopt three complementary metrics to comprehensively assess text-to-CAD generation quality, measuring both code validity and geometric fidelity.

**Chamfer Distance (CD).** Chamfer Distance quantifies the geometric similarity between the generated mesh  $M_g$  and the ground-truth mesh  $M^*$  via bidirectional nearest-neighbor distances between point clouds. Let  $P_g$  and  $P^*$  denote point sets uniformly sampled from  $M_g$  and  $M^*$  respectively:

$$CD(P_g, P^*) = \frac{1}{|P_g|} \sum_p d(p, P^*) + \frac{1}{|P^*|} \sum_q d(q, P_g) \quad (1)$$

We sample 30,000 points per mesh and normalize all models to unit bounding boxes before computation. CD values are reported as  $\times 10^3$  for readability. Lower values indicate better geometric fidelity.

**Invalidity Rate (IR).** Invalidity Rate measures the percentage of generated code samples that fail to produce valid geometry. A sample is considered invalid if: the code fails to execute due to syntax or runtime errors, execution exceeds the 60-second timeout, or the output produces empty or degenerate geometry. Lower IR indicates more reliable code generation. This metric captures whether models can produce syntactically correct and executable CadQuery code, complementing geometric fidelity metrics.

**Intersection over Union (IoU).** IoU measures volumetric overlap between the generated and ground-truth models. We

voxelize both meshes at resolution  $256^3$  and compute:

$$IoU = \frac{|V_g \cap V^*|}{|V_g \cup V^*|} \quad (2)$$

where  $V_g$  and  $V^*$  denote the voxelized representations. IoU captures global shape agreement and is less sensitive to surface sampling noise than CD. Higher values indicate better geometric accuracy.

Importantly, no single metric is sufficient to characterize model performance. CD, IR, and IoU must be analyzed jointly, as a model may generate executable code with poor geometric fidelity, or achieve favorable geometric scores only on a subset of successfully executed samples.

**VLM-based Evaluation.** The above three metrics are designed for L1–L3, where unambiguous ground-truth geometry enables direct geometric comparison. L4, however, targets real-world application scenarios whose design quality extends beyond pure geometric fidelity to include feature completeness and functional suitability. We therefore adopt a complementary VLM-based evaluation protocol: for each generated CAD model, we render 8 multi-view images and submit them to GLM4.6V alongside 5 structured scoring questions—covering overall features, external features, functional features, extended features, and detail features—each rated on a 0–10 scale. An additional Overall score evaluates geometric similarity to the intended design. We also report Invalidity Rate for L4 to capture code executability independently of design quality. The complete scoring rubric is provided in Appendix F.

**Human Verification.** To assess the reliability of VLM-based scoring and the potential for evaluator bias (e.g., same-family preference between the judge and certain candidate models), we conduct human verification on a stratified subset of L4 results. Three annotators with engineering backgrounds independently re-score the subset using the same 0–10 rubric and 8 multi-view renderings provided to the VLM. The overall trend of human ratings is consistent with GLM-4.6V scores, and we observe no systematic same-family preference for GLM 4.7. Full annotator details and agreement statistics are provided in Appendix B.

## 4. Experiments

We conduct comprehensive experiments on Text2CAD-Bench to evaluate state-of-the-art general-purpose large language models and domain-specific models. Our evaluation aims to address three key questions: (1) The current capability of LLMs for text-to-CAD generation. (2) Performance degradation with increasing geometric complexity. (3) Prompt style—geometric or sequential is more effective.

## 4.1. Experimental Setup

### 4.1.1. MODELS

We evaluate two categories of models: general-purpose LLMs and domain-specific models.

**General-purpose LLMs.** We select seven leading large language models that demonstrate strong performance across diverse tasks: **GPT-5.2** (OpenAI et al.), **Claude-4.5-Sonnet** (Anthropic, 2026), **Gemini-3-Flash** (Team et al., 2023), **DeepSeek-V3.2** (DeepSeek-AI et al.), **Qwen3-max** (Yang et al., 2025), **MiniMax-M2.1** (Li et al., 2025), and **GLM-4.7** (Glm et al., 2024). MiniMax-M2.1 is a recent mixture-of-experts model from MiniMax optimized for long-context reasoning and code generation, while GLM-4.7 is the latest iteration of Zhipu AI’s general-purpose foundation model series with reported improvements on code and multi-step reasoning tasks. We include both to broaden coverage of frontier non-Western LLMs alongside the more widely studied GPT, Claude, and Gemini families.

**Domain-specific models.** We evaluate three models specifically designed or fine-tuned for CAD generation: **Text2CAD**(Khan et al.), **Text2CADQuery**(Xie & Ju), and **CADFusion**(Wang et al., 2025).

For large language models, we use official APIs without any hyperparameter tuning to ensure fairness. For smaller domain-specific models, we maintain the default settings from their official GitHub repositories or Hugging Face releases, and provide environments and configurations as close as possible to their original states.

### 4.1.2. IMPLEMENTATION DETAILS

Generated CadQuery code is executed in an isolated Python 3.10 environment with CadQuery 2.4. Successfully executed code produces STEP files, which are converted to point clouds for evaluation using Trimesh. We uniformly sample 30,000 points from each mesh surface. For IoU computation, the voxel resolution is 256. All models are normalized to unit bounding boxes before computing Chamfer Distance. For invalid samples, we do not calculate their CD and IOU.

For the main experiments, we employ a prompt template that provides clear instructions for CadQuery code generation. When generated code fails to execute, we allow up to 3 retry attempts with error feedback. All results reported in Tables 1 and 2 reflect the best output across all attempts—a sample is marked invalid only if all attempts fail. The complete template is provided in F.

## 4.2. Main Results

Table 1 presents the main results across all models and tiers L1 to L3. We report CD, IR and IOU for each experiment,

along with averages weighted by sample count.

### 4.2.1. OVERALL PERFORMANCE

Table 1 presents comprehensive evaluation results across all models and benchmark levels. Several key findings emerge from the analysis. Figure 2 visualizes Chamfer Distance across benchmark levels for general-purpose LLMs. GPT-5.2 achieves the best performance on L1-L2 tasks with the lowest Chamfer Distance (44.31 on L1-Geo) and highest IoU of 0.59. Claude-4.5-Sonnet and DeepSeek-V3.2 form a competitive second tier, while Qwen3-max exhibits weaker geometric reasoning, particularly on complex tasks. Domain-specific models yield substantially higher CD despite lower Invalidity Rates—a finding we discuss below.

**Performance Degradation.** Most models exhibit substantial degradation from L1 to L3. Chamfer Distance increases by 1.3–2.1 $\times$  on average, while Invalidity Rate rises from approximately 15% to 70–90%. The steepest decline occurs between L2 and L3, where advanced features such as sweep, loft, and shell operations cause execution rates to plummet. Notably, Claude-4.5-Sonnet demonstrates the most graceful degradation, achieving the lowest L3-Geo CD (70.13)—surpassing GPT-5.2 (93.46) despite trailing on simpler tasks. Qualitative comparisons illustrating representative generation outputs and failure modes are provided in Appendix G.

**Prompt Style Comparison.** Geometric descriptions outperform sequence descriptions on L1–L2 across multiple metrics. As shown in Figure 2, most models achieve lower CD under geometric prompts on L1–L2 (e.g., GPT-5.2: 44.31 vs. 48.73 on L1), with IoU lines confirming a consistent downward trend from L1 to L3 under both prompt styles. IR further corroborates this pattern: geometric prompts yield 5–10 percentage point lower invalidity rates on L1–L2 across nearly all models (e.g., GPT-5.2: 11.1% vs. 19.5% on L1).

However, this advantage diminishes or reverses on L3. GPT-5.2 (82.94 vs. 93.46), DeepSeek-V3.2 (91.55 vs. 101.23), and MiniMax (79.84 vs. 114.33) all achieve lower CD with sequence prompts, as visible in Figure 2(b). The IR gap also narrows substantially on L3 (e.g., GPT-5.2: 68.0% vs. 74.0%). We attribute this shift to the procedural nature of advanced CAD operations: appearance-based descriptions struggle to convey complex constructive geometry, whereas sequence descriptions directly encode construction logic. This finding suggests that prompt style should be adapted to target geometric complexity in practical text-to-CAD systems.

**General-purpose vs. Domain-specific Models.** A striking divergence emerges between model categories. Text2CAD achieves the lowest IR (as low as 2%) yet the highest CD

## Text2CAD-Bench: A Benchmark for LLM-based Text-to-Parametric CAD Generation

Table 1. Main results on Text2CAD-Bench. CD: Chamfer Distance ( $\times 10^3$ ,  $\downarrow$ ), IR: Invalidity Rate (%), IoU: Intersection over Union ( $\uparrow$ ). Best results are **bolded**

| Model                         | Geometric Prompt (Geo) |                    |                |                 |                    |                |                 |                    |                | Sequence Prompt (Seq) |                    |                |                 |                    |                |                 |                    |                |
|-------------------------------|------------------------|--------------------|----------------|-----------------|--------------------|----------------|-----------------|--------------------|----------------|-----------------------|--------------------|----------------|-----------------|--------------------|----------------|-----------------|--------------------|----------------|
|                               | CD $\downarrow$        | L1 IR $\downarrow$ | IoU $\uparrow$ | CD $\downarrow$ | L2 IR $\downarrow$ | IoU $\uparrow$ | CD $\downarrow$ | L3 IR $\downarrow$ | IoU $\uparrow$ | CD $\downarrow$       | L1 IR $\downarrow$ | IoU $\uparrow$ | CD $\downarrow$ | L2 IR $\downarrow$ | IoU $\uparrow$ | CD $\downarrow$ | L3 IR $\downarrow$ | IoU $\uparrow$ |
| <i>General-purpose LLMs</i>   |                        |                    |                |                 |                    |                |                 |                    |                |                       |                    |                |                 |                    |                |                 |                    |                |
| GPT-5.2                       | <b>44.31</b>           | 11.1%              | <b>0.59</b>    | <b>60.38</b>    | 20.0%              | <b>0.50</b>    | 93.46           | 68.0%              | 0.23           | <b>48.73</b>          | 19.5%              | <b>0.62</b>    | <b>78.21</b>    | 31.6%              | <b>0.47</b>    | 82.94           | 74.0%              | 0.25           |
| Claude-4.5-Sonnet             | 52.62                  | 20.3%              | 0.54           | 71.66           | 43.7%              | 0.48           | <b>70.13</b>    | 70.0%              | 0.25           | 57.08                 | 25.3%              | 0.57           | 75.65           | 53.5%              | 0.46           | 84.82           | 75.0%              | 0.23           |
| DeepSeek-V3.2                 | 53.15                  | 13.3%              | 0.54           | 79.03           | 22.3%              | 0.42           | 101.23          | 69.0%              | 0.17           | 55.55                 | 20.0%              | 0.55           | 97.11           | 32.5%              | 0.37           | 91.55           | 73.0%              | 0.16           |
| Qwen3-max                     | 84.54                  | 21.4%              | 0.40           | 107.40          | 33.4%              | 0.29           | 148.58          | 92.0%              | 0.07           | 68.02                 | 30.1%              | 0.48           | 113.09          | 49.1%              | 0.32           | 97.42           | 93.0%              | 0.12           |
| MiniMax M2.1                  | 65.92                  | 24.8%              | 0.45           | 83.06           | 33.4%              | 0.40           | 114.33          | 91.0%              | 0.26           | 63.63                 | 33.3%              | 0.46           | 106.07          | 43.7%              | 0.32           | <b>79.84</b>    | 89.0%              | <b>0.28</b>    |
| GLM 4.7                       | 67.50                  | 13.7%              | 0.44           | 88.57           | 26.3%              | 0.35           | 92.70           | 83.0%              | 0.25           | 68.24                 | 20.4%              | 0.45           | 100.59          | 34.3%              | 0.33           | 107.32          | 84.0%              | 0.20           |
| Gemini3-Flash                 | 66.82                  | 17.7%              | 0.44           | 88.57           | 12.8%              | 0.35           | 91.61           | 12.8%              | 0.20           | 105.5                 | 16.8%              | 0.31           | 105.47          | 16.8%              | 0.33           | 108.32          | 45.0%              | 0.24           |
| <i>Domain-specific Models</i> |                        |                    |                |                 |                    |                |                 |                    |                |                       |                    |                |                 |                    |                |                 |                    |                |
| Text2CAD                      | 219.57                 | <b>11.0%</b>       | 0.08           | 260.92          | <b>6.0%</b>        | 0.04           | 234.00          | <b>2.0%</b>        | 0.04           | 249.68                | <b>8.0%</b>        | 0.05           | 262.73          | <b>5.0%</b>        | 0.04           | 266.84          | <b>6.0%</b>        | 0.03           |
| CADfusion                     | 229.62                 | 60.0%              | 0.03           | 236.63          | 70.0%              | 0.03           | 209.79          | 53.0%              | 0.03           | 212.17                | 65.0%              | 0.05           | 242.68          | 64.0%              | 0.03           | 191.48          | 51.0%              | 0.03           |
| Text2CADQuery                 | 227.46                 | 44.0%              | 0.07           | 269.11          | 67.0%              | 0.04           | 255.42          | 80.0%              | 0.02           | 229.42                | 29.0%              | 0.07           | 277.28          | 49.0%              | 0.03           | 240.70          | 90.0%              | 0.01           |

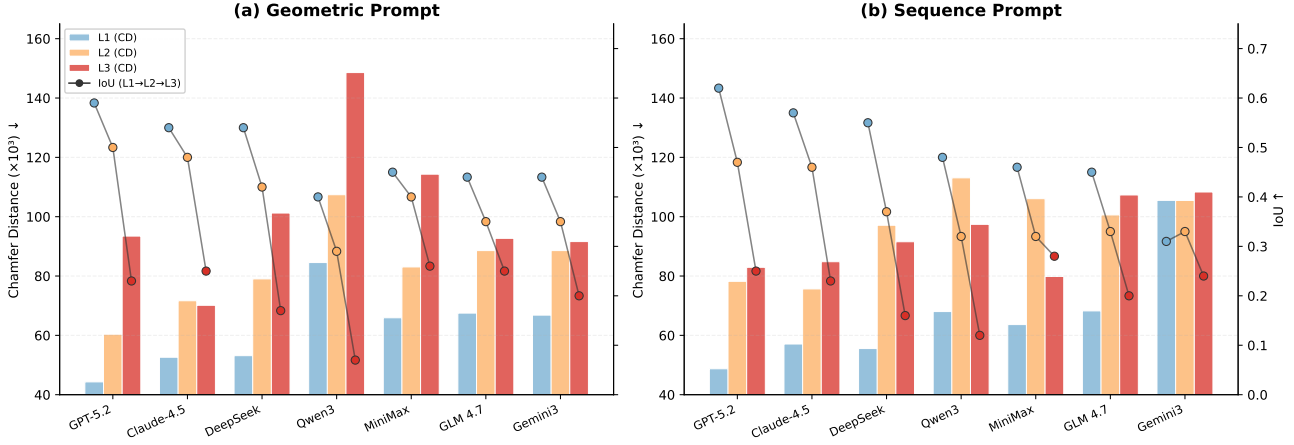


Figure 2. Chamfer Distance (bars, left axis) and IoU (connected markers, right axis) across benchmark levels for general-purpose LLMs under (a) geometric and (b) sequence prompts. Within each model, the IoU line connects L1, L2, and L3 results, visualizing the degradation trend. All models exhibit rising CD and declining IoU from L1 to L3, with Qwen3-max showing the steepest degradation under geometric prompts.

Table 2. L4 evaluation results on real-world application scenarios. IR: Invalidity Rate (%), Q1–Q5 and Overall are scored on a 0–10 scale ( $\uparrow$ ), evaluated by GLM4.6V as judge using 8 multi-view rendered images per sample. Q1: Overall Features, Q2: External Features, Q3: Functional Features, Q4: Extended Features, Q5: Detail Features, Overall: Geometric Similarity. Scores are averaged over valid (successfully executed) samples only. Best results are **bolded**.

| Model             | IR $\downarrow$ | Q1 $\uparrow$ | Q2 $\uparrow$ | Q3 $\uparrow$ | Q4 $\uparrow$ | Q5 $\uparrow$ | Overall $\uparrow$ |
|-------------------|-----------------|---------------|---------------|---------------|---------------|---------------|--------------------|
| GPT-5.2           | 61%             | <b>5.05</b>   | 3.27          | 4.05          | 4.00          | 3.73          | 8.17               |
| Claude-4.5-Sonnet | 54%             | 5.00          | 4.20          | 4.08          | 3.13          | 3.68          | 8.23               |
| DeepSeek-V3.2     | 55%             | 4.94          | 4.20          | 3.90          | 3.31          | 3.46          | <b>8.26</b>        |
| Qwen3-max         | 62%             | 4.43          | 3.37          | <b>4.57</b>   | <b>4.30</b>   | 3.87          | 6.89               |
| MiniMax M2.1      | 81%             | 4.76          | <b>4.63</b>   | 4.26          | 4.53          | <b>4.45</b>   | 7.13               |
| GLM 4.7           | 50%             | 4.86          | 4.54          | 4.46          | 4.32          | 4.01          | 7.02               |
| Gemini3-Flash     | <b>17%</b>      | 4.60          | 3.66          | 3.47          | 3.47          | 3.46          | 8.20               |

(220) and near-zero IoU across all levels and both prompt styles, indicating that it consistently generates executable code without capturing geometric specifications. Notably, this pattern holds under geometric prompts (IR: 11.0%, 6.0%, 2.0% on L1–L3), which differ substantially from

its DeepCAD-derived training representation, suggesting that the low IR reflects a general tendency toward producing syntactically valid but geometrically imprecise outputs rather than distributional alignment with a particular prompt format. This IR–CD divergence underscores the necessity

of multi-metric evaluation: benchmarks relying solely on execution rate risk overestimating models that produce geometrically unfaithful outputs.

### 4.3. Real-World Application Scenarios

Table 2 presents L4 results evaluated by GLM4.6V as judge following the protocol described in Section 3.3.3. Scores are computed only over successfully executed samples.

Several striking patterns emerge that diverge from L1–L3 findings.

**Executability and design quality are largely independent.** Gemini3-Flash achieves the lowest IR (17%) by a wide margin, yet its feature-level scores (Q1–Q5) are among the lowest, indicating that high execution success does not translate to faithful geometry. Conversely, MiniMax exhibits the highest IR (81%) yet achieves the best scores on Q2, Q4, and Q5—its few successful outputs capture application-specific features exceptionally well.

**Geometric precision on L1–L3 does not predict L4 design quality.** GPT-5.2, the strongest model on L1–L3 Chamfer Distance, achieves the highest Q1 score but falls short on external and detail features (Q2, Q5). DeepSeek-V3.2 attains the highest Overall geometric similarity (8.26) but scores below GLM and MiniMax on most feature-level questions.

**Three largely independent capability dimensions.** (1) code executability (reflected by IR), (2) geometric similarity (reflected by Overall), and (3) feature-level design understanding (reflected by Q1–Q5). No single model dominates across all three dimensions, reinforcing the necessity of multi-dimensional evaluation for text-to-CAD systems. This decoupling also suggests that models optimized for code correctness or geometric fidelity on standard parts may require fundamentally different capabilities to handle the contextual richness of real-world design tasks.

### 4.4. Ablation Studies

To validate our choice of output representation, we compare CadQuery against DeepCAD-style command sequences on L1 and L2 subsets, which provide sufficient diversity while maintaining experimental tractability. For command sequence generation, we adopt PythonOCC library built on the OpenSCAD kernel—the same backend that DeepCAD’s domain-specific language (DSL) parses into for execution.

Table 3 presents the results. Under command sequence representation, all models exhibit substantial degradation compared to their CadQuery counterparts (Table 1). IR increases universally—DeepSeek-V3.2 rises from 13.3% to 67.3% on L1-Geo, and GLM 4.7 from 26.3% to 50.7% on L2-Geo. On L2, CD deteriorates severely across all

models (e.g., Gemini3-Flash: 88.57  $\rightarrow$  194.14 under geometric prompts), confirming that compositional geometry is particularly challenging under command sequence representations. Notably, even sequence-style prompts that syntactically align with command sequences fail to match CadQuery performance—Gemini3-Flash achieves CD of 193.45 on L2-Seq versus 88.57 with CadQuery, a  $2.2\times$  degradation. These findings confirm that code-based representations better leverage LLMs’ pretrained capabilities.

**Survivorship bias caveat.** A few cells in Table 3 show CD/IoU *improving* under command sequences despite much higher IR—most notably DeepSeek-V3.2 L1-Geo (IR 13.3%  $\rightarrow$  67.3%, CD 53.15  $\rightarrow$  44.56, IoU 0.54  $\rightarrow$  0.71). Since CD and IoU are computed only over executed samples, a sharp IR rise restricts the evaluation set to the easiest surviving cases and inflates apparent quality; these numbers should not be read as evidence that command sequences are more faithful. The overall conclusion is unaffected, since the dominant pattern in Table 3 is universally higher IR plus worse CD/IoU (e.g., Qwen3-max, Gemini3-Flash on L2-Geo), and Claude-4.5 L2-Geo even shows command sequence *lower* IR yet still worse CD/IoU—ruling out survivorship as the main driver. We recommend reading CD/IoU jointly with IR throughout, and note that matched-subset or worst-case imputation analyses would be a useful addition for future text-to-CAD benchmarks where IR varies substantially across conditions.

## 5. Discussion & Conclusion

We presented Text2CAD-Bench, a benchmark for evaluating text-to-CAD generation capabilities. Through extensive experiments on general-purpose LLMs and domain-specific systems, we arrive at three key conclusions.

**Text-to-CAD remains largely unsolved beyond basic geometry.** While current models achieve reasonable performance on L1-L2 tasks, performance degrades substantially on L3 advanced features. Significant advances in model capabilities or training data are needed before text-to-CAD can reliably support professional workflows.

**Output representation matters critically.** Our comparison demonstrates that Python-based CadQuery representations enable models to exploit pretrained code generation capabilities, yielding substantial improvements over command sequences in both execution validity and geometric accuracy.

**Prompt style influences generation quality.** Geometric descriptions yield better results on L1-L2 tasks, while sequence descriptions show advantages on L3 advanced features. The “global $\rightarrow$ local $\rightarrow$ detail $\rightarrow$ global” pattern aligns well with how language models process spatial information.



Table 3. Output representation comparison: Command Sequence results on L1–L2. CD: Chamfer Distance ( $\times 10^3$ , ↓), IoU (↑), IR: Invalidity Rate (% , ↓).

|     | Model         | L1     |       |      | L2     |       |      |
|-----|---------------|--------|-------|------|--------|-------|------|
|     |               | CD↓    | IR↓   | IoU↑ | CD↓    | IR↓   | IoU↑ |
| Geo | GPT-5.2       | 48.50  | 32.0% | 0.55 | 105.30 | 34.0% | 0.28 |
|     | Claude-4.5    | 55.20  | 42.0% | 0.50 | 125.40 | 38.0% | 0.22 |
|     | DeepSeek-V3.2 | 44.56  | 67.3% | 0.71 | 59.76  | 46.7% | 0.55 |
|     | Qwen3-max     | 168.64 | 49.5% | 0.11 | 219.47 | 71.6% | 0.04 |
|     | GLM 4.7       | 62.90  | 43.0% | 0.42 | 81.90  | 50.7% | 0.31 |
|     | Gemini3-Flash | 64.26  | 16.8% | 0.51 | 194.14 | 29.8% | 0.12 |
| Seq | GPT-5.2       | 45.80  | 38.0% | 0.58 | 112.60 | 52.0% | 0.25 |
|     | Claude-4.5    | 52.40  | 48.0% | 0.52 | 138.70 | 64.0% | 0.18 |
|     | DeepSeek-V3.2 | 44.57  | 55.5% | 0.71 | 66.56  | 46.1% | 0.52 |
|     | Qwen3-max     | 164.23 | 52.7% | 0.16 | 232.87 | 79.3% | 0.05 |
|     | GLM 4.7       | 57.60  | 59.9% | 0.48 | 100.70 | 62.7% | 0.26 |
|     | Gemini3-Flash | 52.62  | 21.4% | 0.56 | 193.45 | 25.5% | 0.11 |

**Capability Decoupling.** Our L4 results reveal that text-to-CAD performance comprises at least three largely independent dimensions: code executability, geometric precision, and design intent understanding. Gemini3-Flash achieves the lowest L4 IR (17%) but scores among the lowest on feature-level questions; MiniMax achieves the highest design quality scores despite an 81% IR; GPT-5.2, the L1–L3 precision leader, scores lowest on L4. This three-way divergence suggests that optimizing for one dimension does not transfer to others, and that future models must address these capabilities jointly rather than in isolation.

**Limitations.** While Text2CAD-Bench covers broader geometric complexity and application domains than existing benchmarks, 600 examples cannot exhaustively represent the full CAD design space. Specialized domains and categories remain underrepresented. We evaluated models without fine-tuning in a single-turn setting. Multi-turn interaction with iterative execution feedback, as well as multi-part assembly generation involving inter-part constraints and mate relationships, remain important directions for future work. Future versions of Text2CAD-Bench will extend beyond single-part solid modeling by incorporating sheet-metal operations, multi-part assemblies with mate constraints, and representation-agnostic evaluation across CadQuery, OpenSCAD, and FreeCAD-style scripts.

**Conclusion.** Text2CAD-Bench provides a challenging testbed for measuring text-to-CAD. Our results establish baselines and reveal advanced CAD features as a significant capability frontier. Part of the benchmark, evaluation code will be released after accept.

## References

- Anthropic. Claude, 2026. URL <https://www.anthropic.com/claude>. Accessed: 2026-01-28.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. d. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F. P., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Guss, W. H., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A. N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., and Zaremba, W. Evaluating large language models trained on code. URL <https://arxiv.org/abs/2107.03374>. Version Number: 2.
- DeepSeek-AI, Liu, A., Feng, B., Wang, B., Wang, B., Liu, B., Zhao, C., Dengr, C., Ruan, C., Dai, D., Guo, D., Yang, D., Chen, D., Ji, D., Li, E., Lin, F., Luo, F., Hao, G., Chen, G., Li, G., Zhang, H., Xu, H., Yang, H., Zhang, H., Ding, H., Xin, H., Gao, H., Li, H., Qu, H., Cai, J. L., Liang, J., Guo, J., Ni, J., Li, J., Chen, J., Yuan, J., Qiu, J., Song, J., Dong, K., Gao, K., Guan, K., Wang, L., Zhang, L., Xu, L., Xia, L., Zhao, L., Zhang, L., Li, M., Wang, M., Zhang, M., Zhang, M., Tang, M., Li, M., Tian, N., Huang, P., Wang, P., Zhang, P., Zhu, Q., Chen, Q., Du, Q., Chen, R. J., Jin, R. L., Ge, R., Pan, R., Xu, R., Chen, R., Li, S. S., Lu, S., Zhou, S., Chen, S., Wu, S., Ye, S., Ma, S., Wang, S., Zhou, S., Yu, S., Zhou, S., Zheng, S., Wang, T., Pei, T., Yuan, T., Sun, T., Xiao, W. L., Zeng, W., An, W., Liu, W., Liang, W., Gao, W., Zhang, W., Li, X. Q., Jin, X., Wang, X., Bi, X., Liu, X., Wang, X., Shen, X., Chen, X., Chen, X., Nie, X., Sun, X., Wang, X., Liu, X., Xie, X., Yu, X., Song, X., Zhou, X., Yang, X., Lu, X., Su, X., Wu, Y., Li, Y. K., Wei, Y. X., Zhu, Y. X., Xu, Y., Huang, Y., Li, Y., Zhao, Y., Sun, Y., Li, Y., Wang, Y., Zheng, Y., Zhang, Y., Xiong, Y., Zhao, Y., He, Y., Tang,

- Y., Piao, Y., Dong, Y., Tan, Y., Liu, Y., Wang, Y., Guo, Y., Zhu, Y., Wang, Y., Zou, Y., Zha, Y., Ma, Y., Yan, Y., You, Y., Liu, Y., Ren, Z. Z., Ren, Z., Sha, Z., Fu, Z., Huang, Z., Zhang, Z., Xie, Z., Hao, Z., Shao, Z., Wen, Z., Xu, Z., Zhang, Z., Li, Z., Wang, Z., Gu, Z., Li, Z., and Xie, Z. DeepSeek-v2: A strong, economical, and efficient mixture-of-experts language model. URL <https://arxiv.org/abs/2405.04434>. Version Number: 5 TLDR: Evaluation results show that, even with only 21B activated parameters, DeepSeek-V2 and its chat versions still achieve top-tier performance among open-source models.
- Glm, T., Zeng, A., Xu, B., Wang, B., Zhang, C., Yin, D., Zhang, D., Rojas, D., Feng, G., Zhao, H., et al. Chatglm: A family of large language models from glm-130b to glm-4 all tools. *arXiv preprint arXiv:2406.12793*, 2024.
- Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., and Steinhardt, J. Measuring mathematical problem solving with the math dataset. *Sort*, 2(4):0–6.
- Hui, B., Yang, J., Cui, Z., Yang, J., Liu, D., Zhang, L., Liu, T., Zhang, J., Yu, B., Lu, K., Dang, K., Fan, Y., Zhang, Y., Yang, A., Men, R., Huang, F., Zheng, B., Miao, Y., Quan, S., Feng, Y., Ren, X., Ren, X., Zhou, J., and Lin, J. Qwen2.5-coder technical report. URL <https://arxiv.org/abs/2409.12186>. Version Number: 3.
- Jayaraman, P. K., Lambourne, J. G., Desai, N., Willis, K. D. D., Sanghi, A., and Morris, N. J. W. SolidGen: An autoregressive model for direct b-rep synthesis. URL <https://arxiv.org/abs/2203.13944>. Version Number: 2 TLDR: This paper presents a new approach that enables learning from and synthesizing B-reps without the need for supervision through CAD modeling sequence data, and demonstrates qualitatively, quantitatively, and through perceptual evaluation by human subjects that SolidGen can produce high quality, realistic CAD models.
- Jun, H. and Nichol, A. Shap-e: Generating conditional 3d implicit functions. *arXiv preprint arXiv:2305.02463*, 2023.
- Khan, M. S., Sinha, S., Sheikh, T. U., Stricker, D., Ali, S. A., and Afzal, M. Z. Text2cad: Generating sequential CAD models from beginner-to-expert level text prompts. URL <https://arxiv.org/abs/2409.17106>. Version Number: 1 TLDR: This work proposes Text2CAD, the first AI framework for generating text-to-parametric CAD models using designer-friendly instructions for all skill levels and introduces a data annotation pipeline for generating text prompts based on natural language instructions for the DeepCAD dataset using Mistral and LLaVA-NeXT.
- Koch, S., Matveev, A., Jiang, Z., Williams, F., Artemov, A., Burnaev, E., Alexa, M., Zorin, D., and Panozzo, D. ABC: A big CAD model dataset for geometric deep learning. In *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 9593–9603. IEEE. ISBN 978-1-7281-3293-8. doi: 10.1109/CVPR.2019.00983. URL <https://ieeexplore.ieee.org/document/8954378/>. TLDR: This work performs a large-scale benchmark for estimation of surface normals, comparing existing data driven methods and evaluating their performance against both the ground truth and traditional normal estimation methods.
- Li, A., Gong, B., Yang, B., Shan, B., Liu, C., Zhu, C., Zhang, C., Guo, C., Chen, D., Li, D., et al. Minimax-01: Scaling foundation models with lightning attention. *arXiv preprint arXiv:2501.08313*, 2025.
- Li, R., Allal, L. B., Zi, Y., Muennighoff, N., Kocetkov, D., Mou, C., Marone, M., Akiki, C., Li, J., Chim, J., et al. Starcoder: may the source be with you! *arXiv preprint arXiv:2305.06161*, 2023.
- Li, X., Sun, Y., and Sha, Z. LLM4cad: Multi-modal large language models for 3d computer-aided design generation. In *International Design Engineering Technical Conferences and Computers and Information in Engineering Conference*, volume 88407, pp. V006T06A015. American Society of Mechanical Engineers.
- Mialon, G., Fourrier, C., Wolf, T., LeCun, Y., and Scialom, T. Gaia: a benchmark for general ai assistants. In *The Twelfth International Conference on Learning Representations*.
- Nichol, A., Jun, H., Dhariwal, P., Mishkin, P., and Chen, M. Point-e: A system for generating 3d point clouds from complex prompts.
- OpenAI, Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., Avila, R., Babuschkin, I., Balaji, S., Balcom, V., Baltescu, P., Bao, H., Bavarian, M., Belgum, J., Bello, I., Berdine, J., Bernadett-Shapiro, G., Berner, C., Bogdonoff, L., Boiko, O., Boyd, M., Brakman, A.-L., Brockman, G., Brooks, T., Brundage, M., Button, K., Cai, T., Campbell, R., Cann, A., Carey, B., Carlson, C., Carmichael, R., Chan, B., Chang, C., Chantzis, F., Chen, D., Chen, S., Chen, R., Chen, J., Chen, M., Chess, B., Cho, C., Chu, C., Chung, H. W., Cummings, D., Currier, J., Dai, Y., Decareaux, C., Degry, T., Deutsch, N., Deville, D., Dhar, A., Dohan, D., Dowling, S., Dunning, S., Ecoffet, A., Eleti, A., Eloundou, T., Farhi, D., Fedus,

- L., Felix, N., Fishman, S. P., Forte, J., Fulford, I., Gao, L., Georges, E., Gibson, C., Goel, V., Gogineni, T., Goh, G., Gontijo-Lopes, R., Gordon, J., Grafstein, M., Gray, S., Greene, R., Gross, J., Gu, S. S., Guo, Y., Hallacy, C., Han, J., Harris, J., He, Y., Heaton, M., Heidecke, J., Hesse, C., Hickey, A., Hickey, W., Hoeschele, P., Houghton, B., Hsu, K., Hu, S., Hu, X., Huizinga, J., Jain, S., Jain, S., Jang, J., Jiang, A., Jiang, R., Jin, H., Jin, D., Jomoto, S., Jonn, B., Jun, H., Kaftan, T., Kaiser, Ł., Kamali, A., Kanitscheider, I., Keskar, N. S., Khan, T., Kilpatrick, L., Kim, J. W., Kim, C., Kim, Y., Kirchner, J. H., Kiros, J., Knight, M., Kokotajlo, D., Kondraciuk, Ł., Kondrich, A., Konstantinidis, A., Kopic, K., Krueger, G., Kuo, V., Lampe, M., Lan, I., Lee, T., Leike, J., Leung, J., Levy, D., Li, C. M., Lim, R., Lin, M., Lin, S., Litwin, M., Lopez, T., Lowe, R., Lue, P., Makanju, A., Malfacini, K., Manning, S., Markov, T., Markovski, Y., Martin, B., Mayer, K., Mayne, A., McGrew, B., McKinney, S. M., McLeavey, C., McMillan, P., McNeil, J., Medina, D., Mehta, A., Menick, J., Metz, L., Mishchenko, A., Mishkin, P., Monaco, V., Morikawa, E., Mossing, D., Mu, T., Murati, M., Murk, O., Mély, D., Nair, A., Nakano, R., Nayak, R., Neelakantan, A., Ngo, R., Noh, H., Ouyang, L., O’Keefe, C., Pachocki, J., Paino, A., Palermo, J., Pantuliano, A., Parascandolo, G., Parish, J., Parparita, E., Passos, A., Pavlov, M., Peng, A., Perelman, A., Peres, F. d. A. B., Petrov, M., Pinto, H. P. d. O., Michael, Pokorny, Pokrass, M., Pong, V. H., Powell, T., Power, A., Power, B., Proehl, E., Puri, R., Radford, A., Rae, J., Ramesh, A., Raymond, C., Real, F., Rimbach, K., Ross, C., Rotsted, B., Roussez, H., Ryder, N., Saltarelli, M., Sanders, T., Santurkar, S., Sastry, G., Schmidt, H., Schnurr, D., Schulman, J., Selsam, D., Sheppard, K., Sherbakov, T., Shieh, J., Shoker, S., Shyam, P., Sidor, S., Sigler, E., Simens, M., Sitkin, J., Slama, K., Sohl, I., Sokolowsky, B., Song, Y., Staudacher, N., Such, F. P., Summers, N., Sutskever, I., Tang, J., Tezak, N., Thompson, M. B., Tillet, P., Tootoonchian, A., Tseng, E., Tuggle, P., Turley, N., Tworek, J., Uribe, J. F. C., Vallone, A., Vijayvergiya, A., Voss, C., Wainwright, C., Wang, J. J., Wang, A., Wang, B., Ward, J., Wei, J., Weinmann, C., Welihinda, A., Welinder, P., Weng, J., Weng, L., Wiethoff, M., Willner, D., Winter, C., Wolrich, S., Wong, H., Workman, L., Wu, S., Wu, J., Wu, M., Xiao, K., Xu, T., Yoo, S., Yu, K., Yuan, Q., Zaremba, W., Zellers, R., Zhang, C., Zhang, M., Zhao, S., Zheng, T., Zhuang, J., Zhuk, W., and Zoph, B. GPT-4 technical report. URL <https://arxiv.org/abs/2303.08774>. Version Number: 6.
- Poole, B., Jain, A., Barron, J. T., and Mildenhall, B. Dream-fusion: Text-to-3d using 2d diffusion. *arXiv preprint arXiv:2209.14988*, 2022.
- Ren, D., Zheng, J., Cai, J., Li, J., and Zhang, J. Extrudenet: Unsupervised inverse sketch-and-extrude for shape parsing. In *European Conference on Computer Vision*, pp. 482–498. Springer, 2022.
- Rukhovich, D., Dupont, E., Mallis, D., Cherenkova, K., Kacem, A., and Aouada, D. CAD-recode: Reverse engineering CAD code from point clouds. URL <https://arxiv.org/abs/2412.14042>. Version Number: 2 TLDR: The proposed CAD-Recode translates a point cloud into Python code that, when executed, reconstructs the CAD model, and shows that the CAD Python code output is interpretable by off-the-shelf LLMs, enabling CAD editing and CAD-specific question answering from point clouds.
- Seff, A., Zhou, W., Richardson, N., and Adams, R. Vitruvion: A generative model of parametric cad sketches. In *The International Conference on Learning Representations (ICLR)*, 2022.
- Team, G., Anil, R., Borgeaud, S., Alayrac, J.-B., Yu, J., Soricut, R., Schalkwyk, J., Dai, A. M., Hauth, A., Millican, K., et al. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.
- Wang, R., Yuan, Y., Sun, S., and Bian, J. Text-to-cad generation through infusing visual feedback in large language models. *arXiv preprint arXiv:2501.19054*, 2025.
- Willis, K. D. D., Pu, Y., Luo, J., Chu, H., Du, T., Lambourne, J. G., Solar-Lezama, A., and Matusik, W. Fusion 360 gallery: a dataset and environment for programmatic CAD construction from human design sequences. 40(4):1–24. ISSN 0730-0301, 1557-7368. doi: 10.1145/3450626.3459818. URL <https://dl.acm.org/doi/10.1145/3450626.3459818>.
- Wu, R., Xiao, C., and Zheng, C. DeepCAD: A deep generative network for computer-aided design models. In *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 6752–6762. IEEE. ISBN 978-1-6654-2812-5. doi: 10.1109/ICCV48922.2021.00670. URL <https://ieeexplore.ieee.org/document/9710909/>. TLDR: This work presents the first 3D generative model for a drastically different shape representation—describing a shape as a sequence of computer-aided design (CAD) operations, and proposes a CAD generative network based on the Transformer.
- Xie, H. and Ju, F. Text-to-CadQuery: A new paradigm for CAD generation with scalable large model capabilities. URL <https://arxiv.org/abs/2505.06507>. Version Number: 1 TLDR: This work proposes generating CadQuery code directly from text, leveraging the strengths of pretrained LLMs to produce 3D models without intermediate representations, using this Python-based scripting language.

- Xu, X., Jayaraman, P. K., Lambourne, J. G., Willis, K. D., and Furukawa, Y. Hierarchical neural coding for controllable CAD model generation. In *International Conference on Machine Learning (ICML)*, pp. 38810–38822. PMLR, a.
- Xu, X., Willis, K. D. D., Lambourne, J. G., Cheng, C.-Y., Jayaraman, P. K., and Furukawa, Y. SkexGen: Autoregressive generation of CAD construction sequences with disentangled codebooks, b. URL <https://arxiv.org/abs/2207.04632>. Version Number: 1 TLDR: SkexGen is presented, a novel autoregressive generative model for computer-aided design (CAD) construction sequences containing sketch-and-extrude modeling operations that generates diverse and high-quality CAD models, enhances user control, and enables efficient exploration of the design space.
- Xu, X., Lambourne, J., Jayaraman, P., Wang, Z., Willis, K., and Furukawa, Y. BrepGen: A b-rep generative diffusion model with structured latent geometry. *ACM Transactions on Graphics (TOG)*, 43(4):1–14, 2024.
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.



## A. Details of Text2CAD-Bench

### A.1. Dataset Statistics

Table 4 summarizes the key statistics of Text2CAD-Bench across complexity levels. We report the average word count for both description styles, average lines of CadQuery code, and average number of API calls in the ground-truth implementations.

Table 4. Statistics of Text2CAD-Bench across complexity levels. We report mean values for description length (word count), code complexity (lines of code), and API usage (number of CadQuery API calls).

| Level             | Geo Words | Seq Words | Code Lines | API Calls |
|-------------------|-----------|-----------|------------|-----------|
| L1 (Basic)        | 89.9      | 81.5      | 7.9        | 10.8      |
| L2 (Intermediate) | 108.4     | 109.6     | 19.1       | 15.0      |
| L3 (Advanced)     | 429.4     | 173.6     | 70.7       | 26.8      |

Several observations emerge from these statistics:

**Complexity gradation is reflected in code structure.** The average lines of code increase substantially from L1 to L3 (7.9  $\rightarrow$  19.1  $\rightarrow$  70.7), representing an approximately 9 $\times$  increase from basic to advanced levels. Similarly, API call counts grow from 10.8 to 26.8, indicating that L3 examples require more sophisticated operation sequences. This progression validates our complexity stratification criteria.

**Description styles exhibit different scaling patterns.** While both geometric and sequence descriptions grow longer with complexity, geometric descriptions scale more dramatically at L3 (429.4 words) compared to sequence descriptions (173.6 words). This divergence reflects the inherent challenge of describing complex freeform geometry in natural language: intricate shapes such as swept surfaces or lofted profiles require extensive spatial elaboration, whereas procedural sequences maintain relative compactness through structured command syntax.

**L1-L2 descriptions are comparable in length.** At lower complexity levels, geometric and sequence descriptions have similar word counts, suggesting that simple geometries can be described with comparable verbosity in either style. The divergence at L3 indicates that description style choice becomes more consequential as geometric complexity increases.

## B. Human Verification Details

This appendix provides additional details on the human verification protocol referenced in Section 3.3.3.

**Annotators.** Three students with engineering backgrounds served as annotators. All had completed introductory CAD coursework and had conducted research in CAD-related areas for over six months, giving them sufficient competence to judge the correspondence between generated models and their textual descriptions from multi-view renderings.

**Scoring interface.** Each annotator was shown the same 8 multi-view renderings provided to the VLM, together with the original text prompt and the 0–10 rubric. Annotators were blind to the source model and to each other’s ratings.

**Results.** The overall trend of human ratings is consistent with the VLM-based scores: models ranked highly by GLM-4.6V on Q1–Q5 and Overall also receive higher human scores, and the relative ordering across the seven candidate models is largely preserved. We did not observe a systematic same-family preference for GLM 4.7 in the human ratings, supporting the use of GLM-4.6V as judge for the full L4 set.

## C. Limitations

While Text2CAD-Bench represents a significant step toward comprehensive evaluation of text-to-CAD generation, we acknowledge several limitations that suggest directions for future work.

### Benchmark Scope

**Domain coverage.** The application domains included in L4 (industrial/mechanical, consumer products, medical devices, architectural elements, and educational models) were selected based on the authors’ assessment of fields where CAD-based manufacturing is prevalent, valuable, or has emerging demand. This selection is inherently non-exhaustive. Specialized

Table 5. Statistics of Text2CAD-Bench.

| Statistic                       | Value |
|---------------------------------|-------|
| Total examples                  | 600   |
| L1                              | 200   |
| L2                              | 200   |
| L3                              | 100   |
| L4                              | 100   |
| Total prompts                   | 1,200 |
| Geometric descriptions          | 600   |
| Sequence descriptions           | 600   |
| <b>Application domains (L4)</b> |       |
| Industrial/Mechanical           | ~40%  |
| Consumer Products               | ~25%  |
| Medical Devices                 | ~15%  |
| Architectural Elements          | ~10%  |
| Educational Models              | ~10%  |

domains such as aerospace, automotive, jewelry design, and microelectronics—each with distinct geometric conventions and precision requirements—remain underrepresented. Future benchmark extensions should incorporate domain experts to ensure broader and more representative coverage.

**Geometric categories.** Our benchmark focuses primarily on solid modeling operations. Important CAD categories such as sheet metal design, multi-part assemblies, and parametric families with complex constraints are not covered. These categories present unique challenges (e.g., bend allowance calculations, mate constraints) that warrant dedicated evaluation.

**Sample size.** With 600 examples, Text2CAD-Bench cannot exhaustively represent the full space of CAD design tasks. While we prioritize quality and diversity over quantity, certain geometric configurations and edge cases may be underrepresented.

### Representation Choice

We exclusively adopt CadQuery as our target representation. While this choice is well-motivated (Section 2.2), it limits direct comparison with systems targeting other CAD formats such as OpenSCAD, FreeCAD scripts, or proprietary formats (e.g., SolidWorks macros, AutoCAD LISP). Models optimized for alternative representations may exhibit different capability profiles that our benchmark does not capture.

### Evaluation Protocol

We evaluated models exclusively in zero-shot and few-shot settings without fine-tuning. Models specifically trained on our benchmark distribution might achieve substantially different results. The relative ranking of models could shift significantly under fine-tuning regimes. Besides, we did not explore multi-turn interaction, where iterative refinement based on execution feedback or visual inspection could potentially improve generation quality. Such interactive workflows may better reflect practical text-to-CAD usage patterns.

The rapid pace of LLM development means that evaluation results can quickly become outdated. New models with improved code generation capabilities are released frequently, and existing models receive continuous updates. To address this challenge, **we establish a public leaderboard** alongside our benchmark release. We encourage researchers and practitioners to evaluate their models on Text2CAD-Bench and submit results to the leaderboard, fostering continuous benchmarking and community engagement. The leaderboard will be maintained at [URL] and accepts submissions following the standardized evaluation protocol described in Section 4.1.

## D. Prompts Used in Text2CAD-Bench

This appendix provides the complete prompt templates used in our evaluation. All experiments employ a two-stage prompting strategy: an initial generation prompt and, upon execution failure, a retry prompt that incorporates error feedback.

### D.1. System Prompt

The system prompt establishes the model’s role as a CadQuery expert and provides critical API guidance to prevent common errors. We include five diverse examples spanning different geometric operations (chamfer, sweep, loft, twist extrude, and fillet) to demonstrate proper CadQuery usage patterns.

For few-shot experiments, we augment the base prompt with additional input-output demonstrations. These examples are drawn from a held-out set that shares the same data source as the evaluation benchmark but contains no overlapping samples. The selected demonstrations cover diverse advanced geometric features (e.g., sweep, loft, shell, complex boolean operations), enabling models to learn correct API usage patterns for operations that may be underrepresented in their pretraining corpora. Specifically, we prepend  $k$  demonstration pairs before the target task description, where each pair consists of a natural language description and its corresponding ground-truth CadQuery implementation.

You are an expert CadQuery programmer. Generate Python code using CadQuery to create 3D models.

CRITICAL RULES:

1. MUST start with: `import cadquery as cq`
2. MUST assign final result to variable named `'result'`
3. DO NOT use `show_object()`, `display()`, or any visualization
4. DO NOT include explanations, ONLY code in ````python```` block

CADQUERY API NOTES (IMPORTANT - avoid common errors):

- There is NO `.cone()` method. For cones, use:  
`cq.Solid.makeCone(radius1, radius2, height)`
- There is NO `.array()` method. Use  
`.rarray(xSpacing, ySpacing, xCount, yCount)` for rectangular arrays
- For circular patterns, use  
`.polarArray(radius, startAngle, angle, count)`
- `.fillet()` and `.chamfer()` require a solid first.  
Always create geometry before filleting.
- Use `.box(length, width, height)` for boxes
- Use `.cylinder(height, radius)` for cylinders
- Use `.sphere(radius)` for spheres
- Chain operations properly:  
`cq.Workplane("XY").box(10,10,10).faces(">Z").hole(2)`
- For extrusion: `.extrude(height)`
- For cutting: `.cut(other_solid)`
- For union: `.union(other_solid)`

[Five in-context examples omitted for brevity;  
see supplementary materials for complete prompt]

OUTPUT FORMAT:

```
```python
import cadquery as cq
# your code here
result = ... # final CadQuery Workplane or Shape
```
```

#### D.1.1. IN-CONTEXT EXAMPLES

The system prompt includes five carefully designed examples that demonstrate diverse CadQuery operations:

**Chamfered Hexagonal Prism.** Demonstrates polygon creation and edge chamfering with face selection.

**Helical Spring.** Demonstrates sweep operation along a programmatically generated helix path.

**Circle to Square Loft.** Demonstrates loft operation between dissimilar cross-sections.

**Twisted Cycloid Gear.** Demonstrates parametric curves and twist extrusion for complex geometry.

**L-Bracket with Fillet and Chamfer.** Demonstrates profile sketching and combined fillet/chamfer operations with edge selection.

These examples were selected to cover operations frequently required in L2-L3 tasks while avoiding overlap with evaluation samples. For each evaluation sample, the user prompt simply provides the task description:

```
{description}
```

where {description} is replaced with either the geometric or sequence description from our benchmark.

## D.2. Retry Prompt

When generated code fails to execute, we provide the model with error feedback and request a corrected version. This retry mechanism allows models to leverage error messages for self-correction.

The previous CadQuery code attempt failed with an error.  
Please analyze the error and generate corrected code.

```
## Original Task
{original_prompt}

## Previous Code Attempt (Iteration {iteration}):
```python
{previous_code}
```

## Error Message:
{error_message}

## Instructions:
1. Carefully analyze the error message above
2. Identify the root cause of the failure
3. Generate corrected CadQuery code that fixes the issue
4. Ensure the code follows all CadQuery API rules
   mentioned in the system prompt
5. Output ONLY the corrected code in a ```python``` block
```

Please generate the corrected code:

## E. Details in Building Text2CAD-Bench

The construction of Text2CAD-Bench follows a rigorous human-in-the-loop pipeline designed to ensure quality, consistency, and geometric diversity. This appendix provides additional details on our collaborative authoring process and quality control procedures.

Our benchmark construction involved a team of three contributors with backgrounds in mechanical engineering and CAD modeling. The construction process spanned approximately four weeks, with each example undergoing multiple iterations before acceptance into the final benchmark.

For each example, the authoring process begins with a human designer specifying the target geometry, intended difficulty level, and key geometric features to be incorporated. The designer first composes the natural language descriptions—both geometric and sequence styles—that fully specify all parameters of the intended model. This description-first approach ensures that the textual specification serves as the authoritative reference, preventing post-hoc rationalization of implementation details.

Following description authoring, the designer develops the corresponding CadQuery implementation, often consulting AI assistants (Claude, Gemini and GPT) for API syntax, debugging assistance, or alternative implementation strategies. This collaborative approach leverages AI efficiency for routine coding tasks while maintaining human oversight for geometric correctness and difficulty calibration. Importantly, AI assistants serve as coding aids rather than autonomous generators—all geometric decisions and final code validation remain under human control.

For geometric descriptions, authors compose natural language following the “global shape → local features → fine details → global summary” pattern established in Section 3.2. For sequence descriptions, we adopt a simplified specification format compared to verbose coordinate-based representations. For instance, rather than specifying a rectangle by its four corner coordinates (e.g., “draw lines from (0,0) to (80,0) to (80,50) to (0,50) and back to origin”), we describe it by center point, orientation, and dimensions (e.g., “In XY plane, sketch a rectangle centered at the origin with length 80mm and width 50mm”). This simplification reduces description verbosity while preserving complete geometric specification, and better reflects how practitioners naturally communicate design intent.

The iterative refinement process typically requires 2–5 iterations per example. Common revision triggers include: execution errors due to API misuse, geometric output deviating from design intent, code complexity misaligned with target difficulty level, and insufficient feature coverage for the designated tier. Each iteration involves code modification, execution validation, and visual inspection of the generated mesh against the intended geometry. An independent reviewer (not the original author) verifies each example against four criteria: (1) code executes without errors or warnings in the standardized environment; (2) generated geometry accurately reflects both description styles; (3) geometric complexity aligns with the assigned difficulty level; (4) geometric and sequence descriptions are mutually consistent,



describing identical geometry. Examples failing any criterion are returned for revision or excluded from the benchmark.

To ensure consistency across contributors, we established detailed annotation guidelines specifying description conventions, difficulty classification criteria, and common pitfalls to avoid. Regular calibration sessions were conducted to align reviewers' judgments on edge cases, particularly for examples near difficulty level boundaries.

## F. Representative Prompt-Code Pairs

We provide representative examples for each benchmark level to illustrate the progression in geometric complexity, code structure, and description style.

### L1 (Basic)

**Geometric Prompt:** *“The model is a prismatic solid based on a regular hexagon, with uniform material throughout. The base profile is a regular hexagon with an inscribed circle diameter of 50mm, centered at the origin. The hexagonal prism is extruded 20mm in the positive Z-axis direction. At the center of the top surface, there is a circular through-hole with a diameter of 20mm. The six outer edges on the top surface are chamfered at 45 degrees with a chamfer width of 2mm.”*

**Sequence Prompt:** *“Set the XY plane as the base sketch plane. Draw a regular hexagon centered at the origin with an inscribed circle diameter of 50.0mm. Extrude 20.0mm in the +Z direction. On the top face, draw a concentric circle with diameter 20.0mm and perform an extruded cut through all. Apply a 2.0mm chamfer to the six outer edges of the top face.”*

**Ground-Truth CadQuery Code:**

```
from math import cos, pi
import cadquery as cq
result = (
    cq.Workplane("XY")
    .polygon(6, 50 / cos(pi/6))
    .extrude(20)
    .faces(">Z").workplane()
    .circle(10)
    .cutThruAll()
    .faces(">Z").edges("%Line")
    .chamfer(2)
)
```

L1 requires single primitives with basic finishing features (chamfer, through-hole). The code is concise (under 10 lines) with a straightforward operation chain.

### L2 (Intermediate)

**Geometric Prompt:** *“The overall appearance is a hemisphere with a cross-shaped recess. The main base feature is a hemisphere with a radius of 40mm. On the flat side, a deep cross-shaped groove is cut with a width of 10mm and depth of 20mm. The ends of the groove are open. The cross-shaped groove divides the hemisphere into four quadrant-like segments connected at the bottom.”*

**Sequence Prompt:** *“Draw a semicircle with radius 40mm and rotate 180 degrees to generate a hemisphere. On the top plane, draw two rectangles that are concentric and perpendicular; both with width 10mm and lengths extending beyond the hemisphere edge. Extruded cut downward by 20mm to form a cross-shaped groove.”*

**Ground-Truth CadQuery Code:**

```
import cadquery as cq
sphere_radius = 40
slot_width = 10
slot_depth = 20
slot_length = 100
hemisphere = (
    cq.Workplane("XY")
    .sphere(sphere_radius)
    .cut(
        cq.Workplane("XY")
        .transformed(offset=(0, 0, -sphere_radius))
        .box(sphere_radius * 3, sphere_radius * 3,
            sphere_radius * 2, centered=True)
    )
)
```

)

L2 introduces boolean operations (sphere-box subtraction for hemisphere construction, cross-shaped cut) and requires correct sequencing of multiple operations.

### L3 (Advanced)

**Geometric Prompt:** “The overall shape is a slender spindle with cross-sections smoothly transitioning from circle to ellipse and back. The spine is controlled by a cubic Bezier curve with control points  $P_0(0,0,0)$ ,  $P_1(30,20,0)$ ,  $P_2(100,25,0)$ ,  $P_3(150,5,0)$ . Four cross-sections:  $X=0$  circle  $d=15\text{mm}$ ,  $X=50$  ellipse  $25 \times 20\text{mm}$ ,  $X=100$  ellipse  $22 \times 18\text{mm}$ ,  $X=150$  circle  $d=20\text{mm}$ . Shell wall thickness  $1.5\text{mm}$ .”

**Sequence Prompt:** “Create four datum planes at  $X=0,50,100,150$ . Draw respective circle/ellipse profiles on each. Draw a Bezier guide line with control points  $P_0$ – $P_3$ . Loft through all sections with the guide line. Shell with  $1.5\text{mm}$  wall thickness, removing both end faces.”

**Ground-Truth CadQuery Code (abbreviated):**

```
import cadquery as cq
from math import isclose

def solve_cubic_bezier_z_at_x(x_target, p0, p1,
                              p2, p3, tolerance=1e-4):
    t_min, t_max = 0.0, 1.0
    for _ in range(100):
        t = (t_min + t_max) / 2
        x_val = ((1-t)**3*p0[0] + 3*(1-t)**2*t*p1[0]
                 + 3*(1-t)*t**2*p2[0] + t**3*p3[0])
        if isclose(x_val, x_target, abs_tol=tolerance):
            break
        elif x_val < x_target:
            t_min = t
        else:
            t_max = t
    ...
arm_solid = (cq.Workplane("YZ")
             .add(wires).toPending()
             .loft(ruled=False))
result = arm_solid.faces("<X or >X").shell(-1.5)
```

L3 demands parametric curve computation, multi-profile loft across dissimilar cross-sections, and shell operations. Code complexity increases substantially ( $\sim 70$  lines, 26+ API calls), reflecting the qualitative leap in geometric reasoning required.

### L4 (Real-World)

L4 examples target real-world application scenarios and are evaluated via VLM-based scoring rather than geometric comparison with ground-truth code. We provide one representative prompt to illustrate the contextual richness of L4 tasks.

**Prompt:** “A modular cable management desktop station designed for neatly managing office cables and providing a phone stand. The device is approximately 12 cm tall, with an 8 cm diameter circular base and a layered design. The bottom layer features three 1.5 cm wide semi-circular grooves for securing charging cables of different thicknesses. The middle layer is a 360-degree rotating tray with six 2 cm diameter holes for inserting and retrieving various USB cable heads, preventing cables from slipping off the desk. The top of the device is designed as a 65-degree angled phone holder with an array of ventilation holes on the back panel and a non-slip pad at the bottom.”

Unlike L1–L3, L4 descriptions embed domain-specific conventions and functional requirements (e.g., cable routing, phone viewing angle, ventilation) that models must interpret within application context. The geometric complexity of L4 examples varies, but the primary challenge lies in translating contextual design intent into correct parametric geometry.

## G. Qualitative Results

We provide qualitative comparisons across representative samples to illustrate generation quality and characteristic failure modes at each benchmark level.

Figure 3 presents L1–L2 results under both prompt styles. Several patterns emerge: even basic L1 tasks cause execution failures for multiple models, successful execution does not guarantee geometric correctness, and prompt style affects generation differently across models—GPT fails on L1\_109 with geometric prompts but succeeds with sequence prompts, while Claude shows the opposite pattern.

Figures 4 show L3 results under geometric and sequence prompts, respectively. Advanced operations such as sweep, loft, and shell lead to widespread execution failures, with only a few models producing partial outputs on selected examples. These visualizations confirm that the high L3 invalidity rates reported in Table 1 reflect fundamental capability limitations rather than isolated errors.

Figure 5 visualizes representative L4 outputs across diverse application domains. Since L4 is evaluated via VLM-based scoring (Section 3.3.3) rather than geometric comparison with ground truth, ground-truth models are not shown.

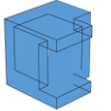
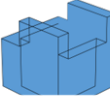
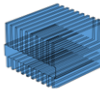
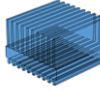
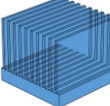
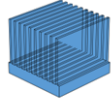
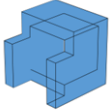
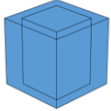
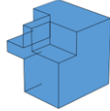
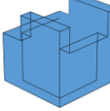
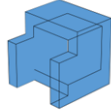
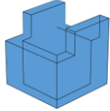
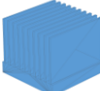
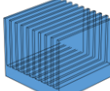
|            | Gt     | Claude                                                                              | Deepseek                                                                            | MiniMax                                                                             | Qwen                                                                                | Gemini                                                                               | GPT                                                                                   | GLM                                                                                   |                                                                                       |
|------------|--------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| Geo_prompt | L1_109 |    |    |    | ✗                                                                                   | ✗                                                                                    |    |    | ✗                                                                                     |
|            | L1_113 |    |    |    |    |     |    |    |    |
|            | L2_14  |    |    |    |    |    |    |    | ✗                                                                                     |
|            | L2_44  |    |    |    |    |     |    |    |    |
|            | L2_137 |    |    |    |    |     |    |    |    |
| Seq_prompt | L1_109 |  | ✗                                                                                   |  | ✗                                                                                   | ✗                                                                                    |  |  | ✗                                                                                     |
|            | L1_113 |  | ✗                                                                                   |  |  |   |  | ✗                                                                                     |  |
|            | L2_14  |  |  |  |  |  |  |  |  |
|            | L2_44  |  | ✗                                                                                   |  | ✗                                                                                   | ✗                                                                                    |  | ✗                                                                                     |  |
|            | L2_137 |  |  |  | ✗                                                                                   | ✗                                                                                    |  |  |  |

Figure 3. Qualitative comparison of generated CAD models on L1–L2 examples under geometric (top) and sequence (bottom) prompts. Red crosses indicate execution failures.

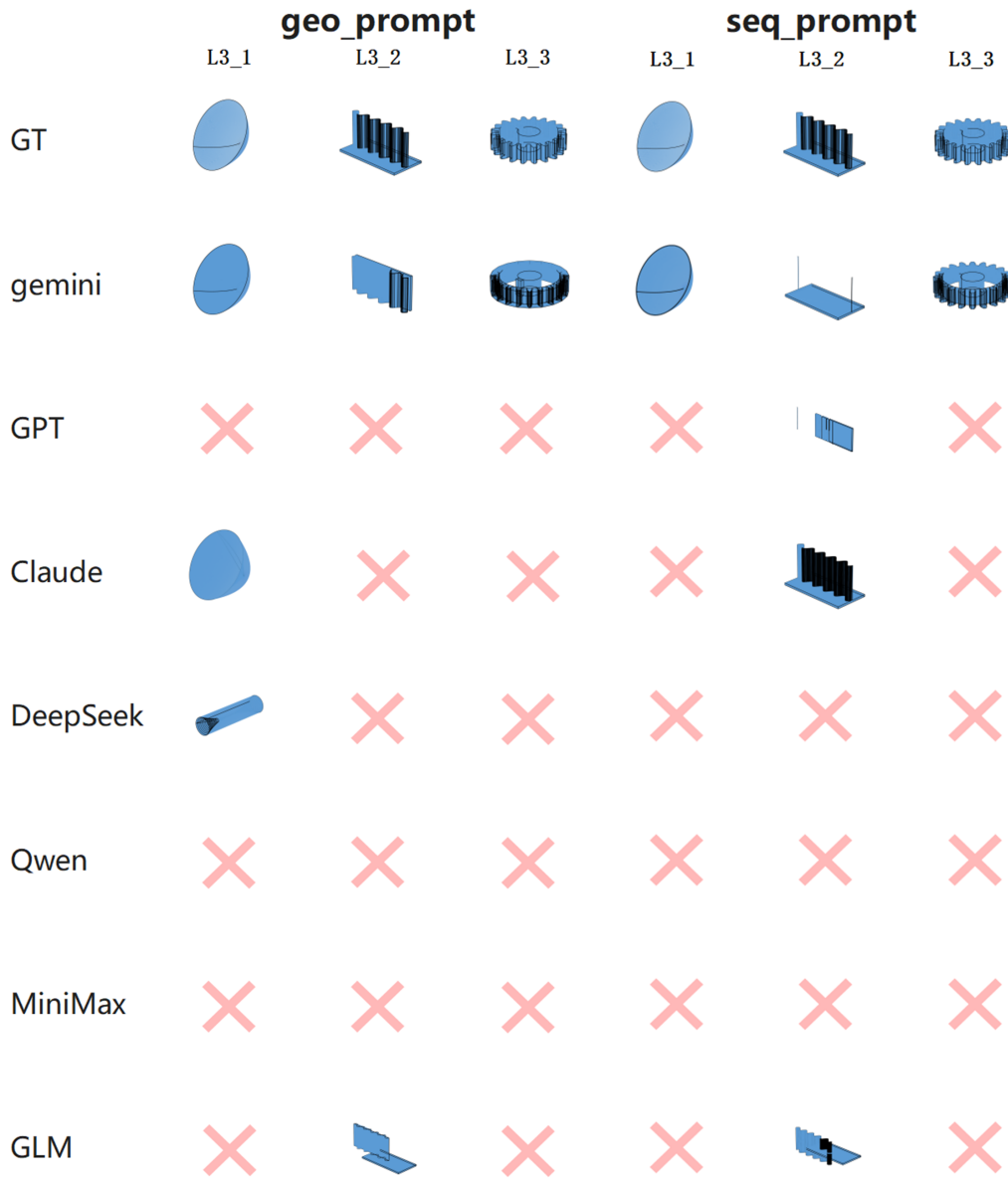


Figure 4. Qualitative results on L3 under geometric prompts.



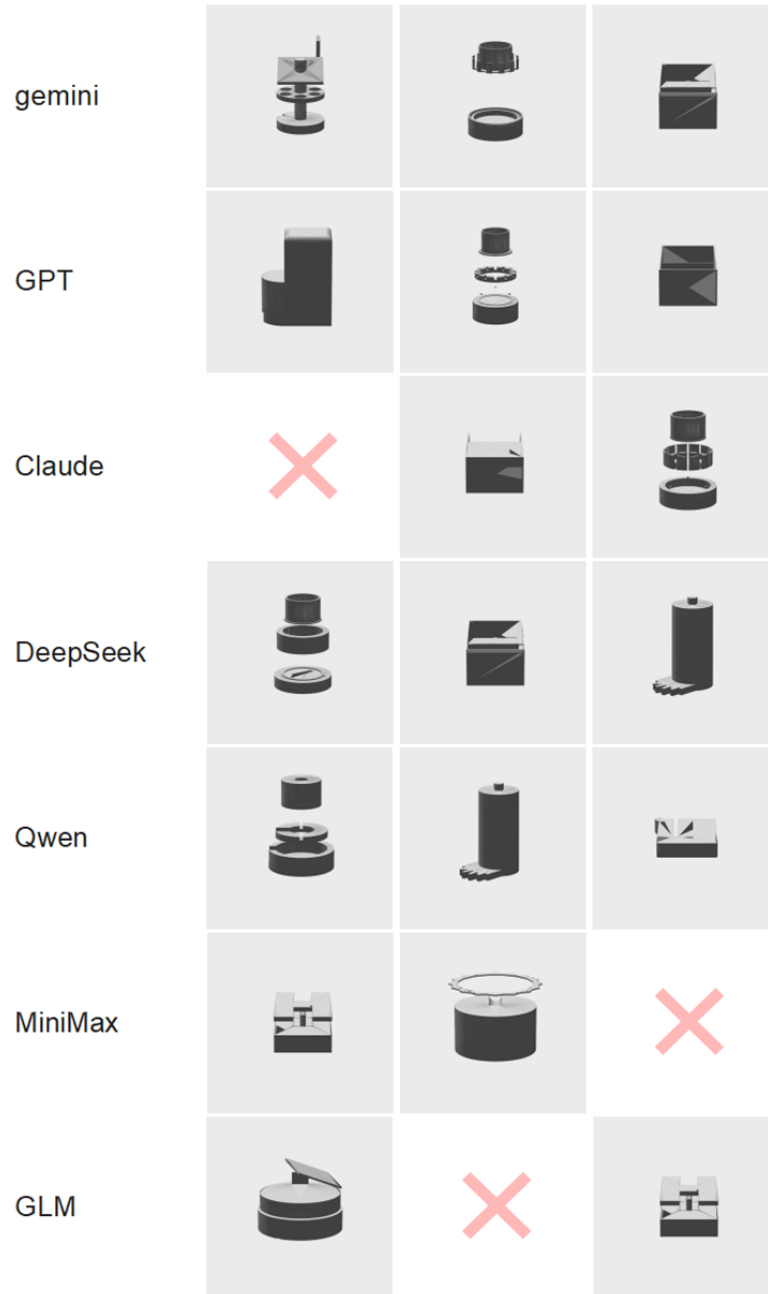


Figure 5. Qualitative results on L4 (Real-world) examples across diverse application domains. Ground-truth models are not shown as L4 evaluation relies on VLM-based scoring rather than geometric comparison.